

Rotman

INTRO TO DATA VISUALIZATION

Part IV Build Dashboards with Quarto and Plotly Express

October 2, 2026 Prepared by Jay Cao / [MDAL](#)

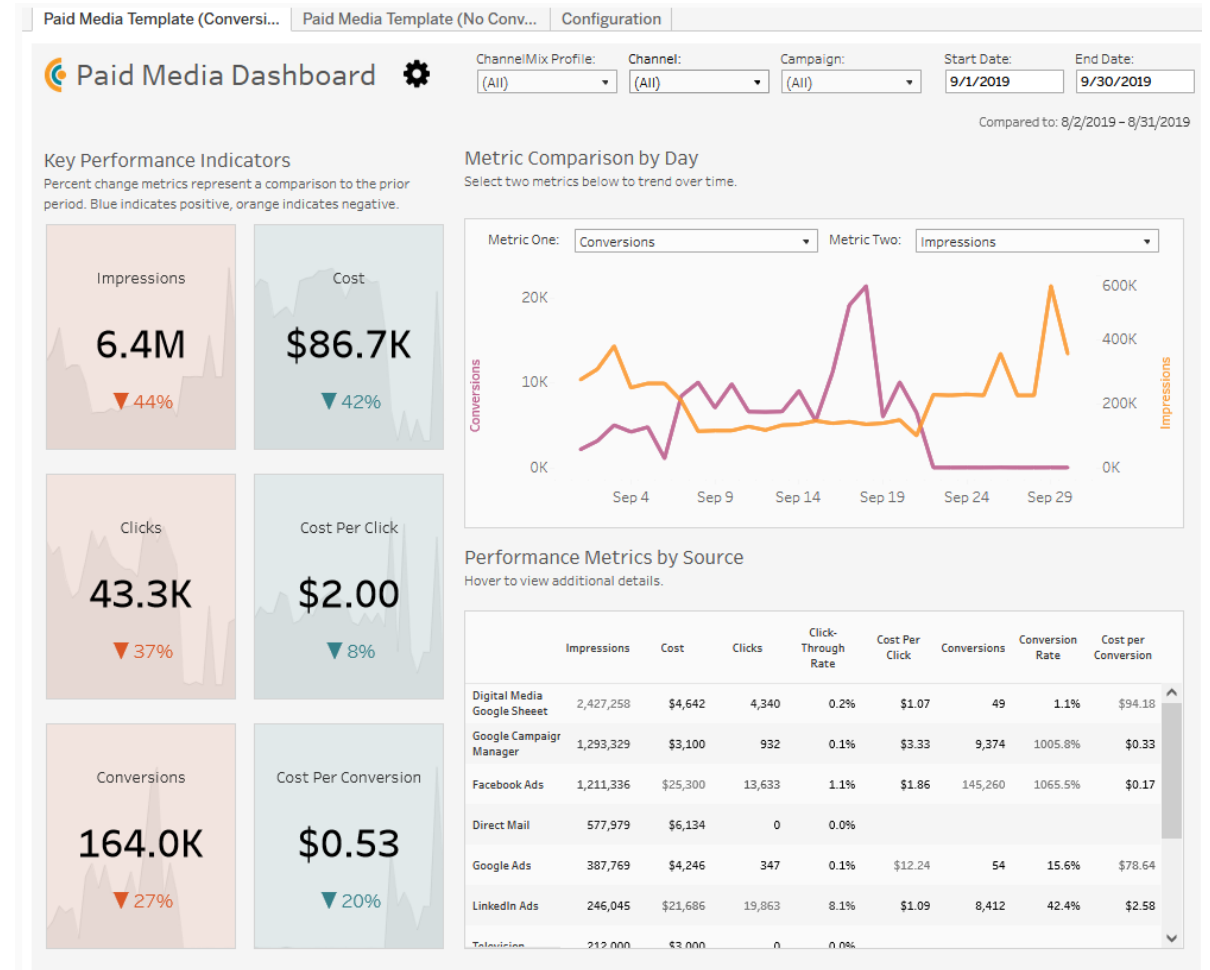
Website: <https://rmdal.github.io/mma-dv-2026/>



Rotman School of Management
UNIVERSITY OF TORONTO

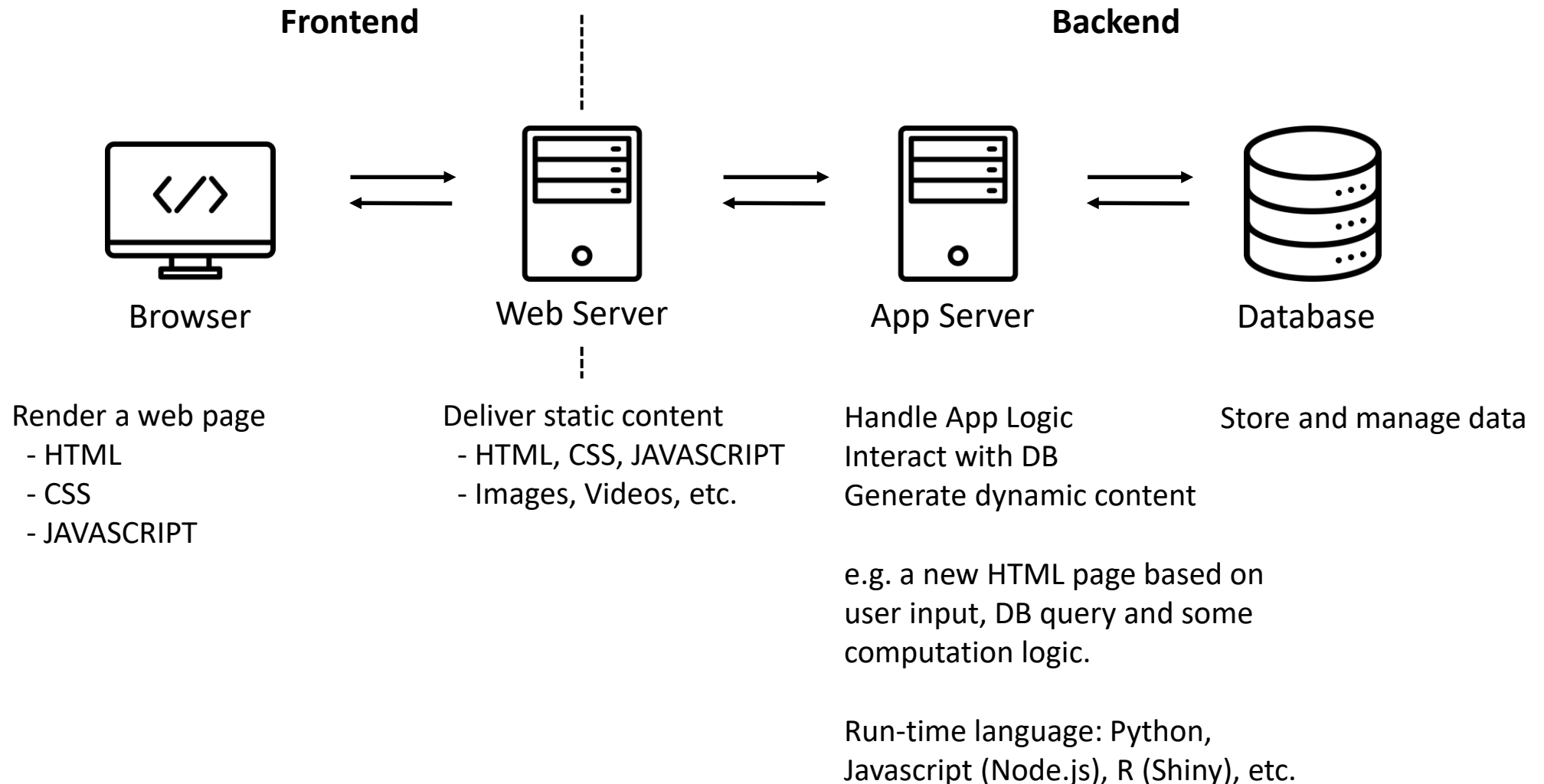
What is a Dashboard

- A way to display related data visualization and summary in one place
- Usually contains interactive and dynamic features
- Usually accessible via a web browser
- Market campaign performance example
 - Key Performance Indicators (KPIs)
 - Metric comparison line plot
 - Aggregated data table
 - Interactivity (data filters, etc.)



Source: [Paid Media Dashboard](#)

Most Dashboards are Web Apps

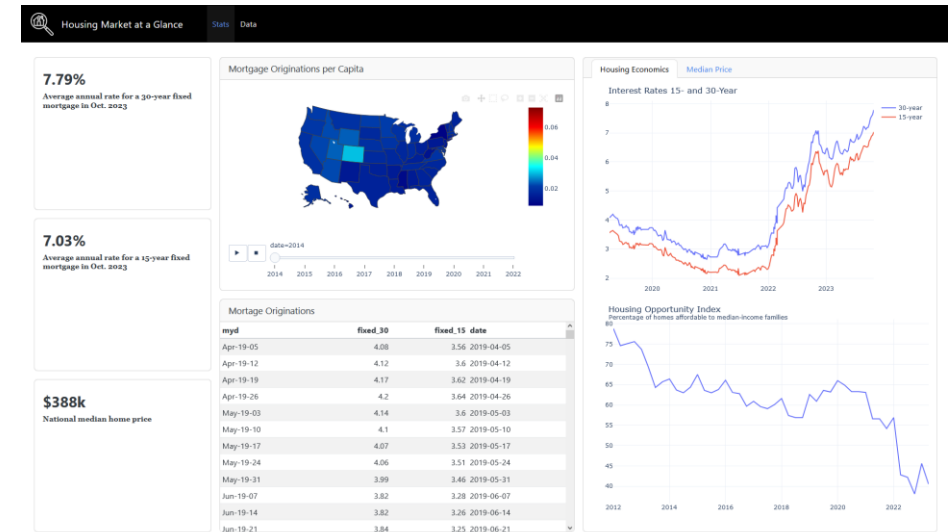


Python Dashboard Tooling

- Purpose of those tools
 - Make dashboard building easy
 - Require minimum knowledge on web technology (html, css, javascript, etc.)
- Landscape
 - [Dash](#) (by [Plotly](#))
 - [Panel](#) (by [Anaconda](#))
 - [Voila](#) (a subproject of Project Jupyter by [Quantstack](#))
 - [Streamlit](#)
 - [Gradio](#) (specialize in ML apps)
 - [Shiny for Python](#) (by [Posit](#))
 - [Quarto Dashboards](#) (by Posit)

Quarto Dashboards

- Easy to use (Markdown + Python or R)
 - IMO, truly need little knowledge of web dev
- Support Python and R
- Support simple interactivity (via Javascript)
 - can be deployed as static web pages
- Support enhanced interactivity (via Shiny for Python)
 - need to be deployed with Shiny Server
- Downside
 - Less flexible for advanced dashboard builders



An example of Quarto dashboard with simple interactivity

<https://ivelasq.github.io/mortgage-dashboard/>

Simple Interactivity Dashboard

- Dashboard that need no backend to run (i.e. no app server, DB, etc.)
 - Sometimes called “static” dashboard
- Use Javascript for basic interactivity (e.g., simple data filter)
 - With Quarto, you don’t need to know how to code in Javascript
- Need only a web server to make it available to the internet
 - For example, you can host it on Github for free
- Quarto Dashboard examples
 - <https://jjallaire.github.io/stock-explorer-dashboard/> (note the web host)

Enhanced Interactivity Dashboard

- Dynamically retrieve data and display/visualize information
 - E.g., real-time update, complex underlying data transformation & analytics, etc.
- Support complex interactivity
 - E.g., user-based interactivity, user inputs that trigger backend business logics, etc.
- Require App server, DB, etc. in addition to a Web server
- Quarto Dashboard Examples
 - <https://jjallaire.shinyapps.io/penguins-dashboard/> (note the web host)

Simple Db – Code Skeleton 1

```
---
title: "Superstore"
format:
  dashboard:
    logo: super.png
---

```{python}
load dataset, prepare it for display and plot
```

# Sales (`{python} year`) → Level 1 heading: page

## Row {height=15%} → Level 2 heading: row

```{python}
#| content: valuebox
#| title: "Total Sales"
```

```{python}
#| content: valuebox
#| title: "Total Profit"
```
```

YAML Header

Python Code

Value Box: Display Python Variable

Value Box: Display Python Variable



Simple Db – Code Skeleton 2

```
## Row {height=35%} → Level 2 heading

```{python}
#| title: Sales by State
#| padding: 0

sales by state plot
```

## Row {height=35%} → Level 2 heading

```{python}
#| title: Sales by Segment
#| padding: 0

sales by segment plot
```

## Row {height=35%} → Level 2 heading

```{python}
#| title: Sales by Category
#| padding: 0

sales by category plot
```

## Row {height=35%} → Level 2 heading

```{python}
#| title: Sales by Sub-Category
#| padding: 0

sales by sub-category plot
```
```

Content Card: Python Plot

Content Card: Python Plot

Content Card: Python Plot

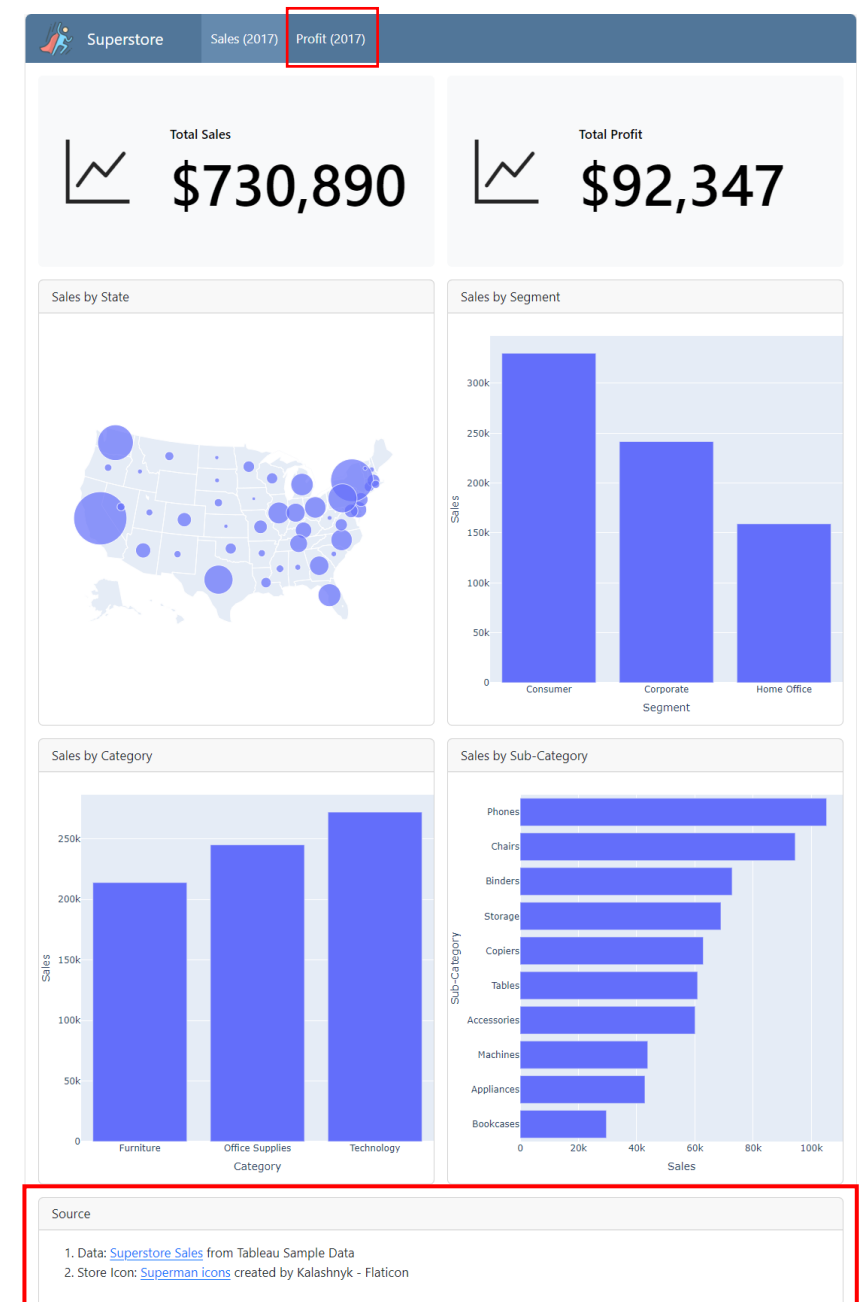
Content Card: Python Plot



Simple Db – Code Skeleton 3

```
## Row {height=15%} → Level 2 heading: row  
::: {.card title="Source"}  
:::  
# Profit (`{python} year`) → Level 1 heading: page  
  
**It's Your Turn to Build.**
```

```
> quarto render superstore.qmd
```



Enhanced Dashboard – Code Skeleton 1

```
---
title: "Superstore V2"
format:
  dashboard:
    logo: super.png
→ server: shiny
---

```{python}
load python library (shiny, plotly.express, etc.)

load and prepare data

```

→ ## {.sidebar}
```{python}
ui.input_select()

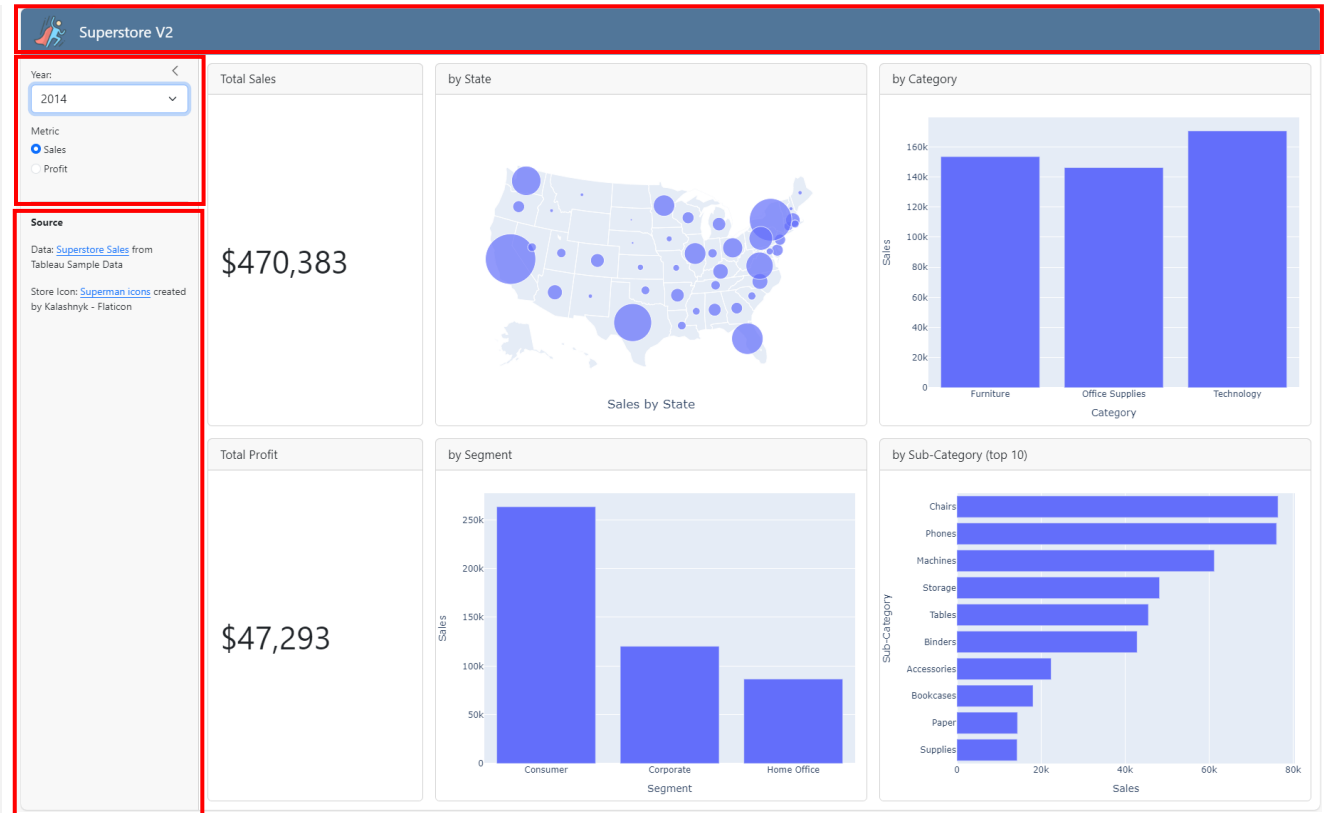
ui.input_radio_buttons()
```

---

**Source**

Data: [Superstore Sales](https://public.tableau.com/app/learn/sample-data)
from Tableau Sample Data
```

} Python Code
for Inputs



Enhanced Dashboard – Code Skeleton 2

```
'''{python}
→ @reactive.calc
def ss_year():
    return ss[ss["Ship
    Date"].dt.year==int(input.year())]

Re-calculate when
inputs change

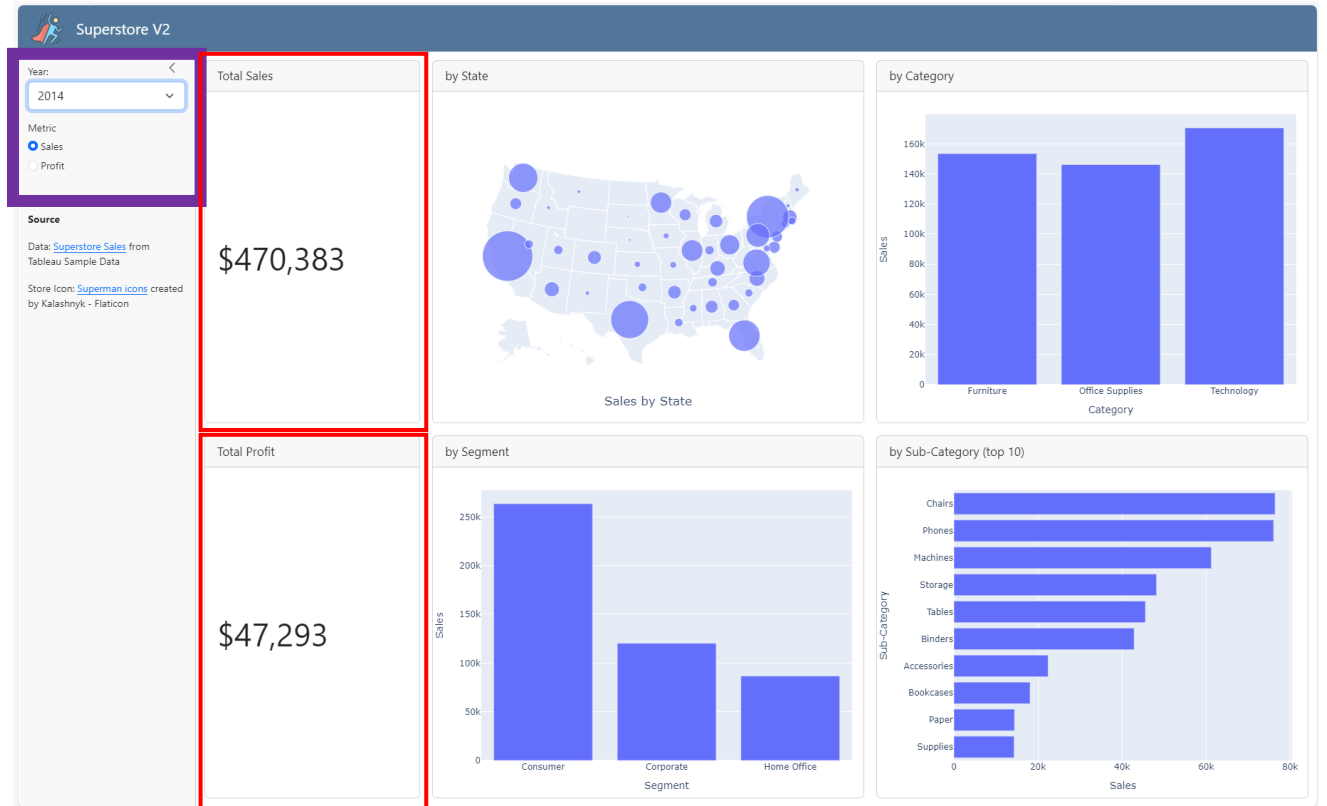
→ @reactive.calc
def sales():
    return ss_year()["Sales"].sum()

# more reactive calculation
'''

## Column {width=20%}
'''{python}
#| title: Total Sales
#| padding: 0
→ @render.ui
def sales_value_box():
    return shiny.ui.value_box()

'''

'''{python}
#| title: Total Profit
#| padding: 0
→ @render.ui
def profit_value_box():
    return shiny.ui.value_box()
'''
```



Enhanced Dashboard – Code Skeleton 3

```
## Column {width=40%}
```

```
```{python}
#| title: by State
#| padding: 0
```

```
→@render_widget
def plot_by_state():
 fig = px.scatter_geo()
 fig.layout.update()
 return fig
```
```

Re-render when
inputs change

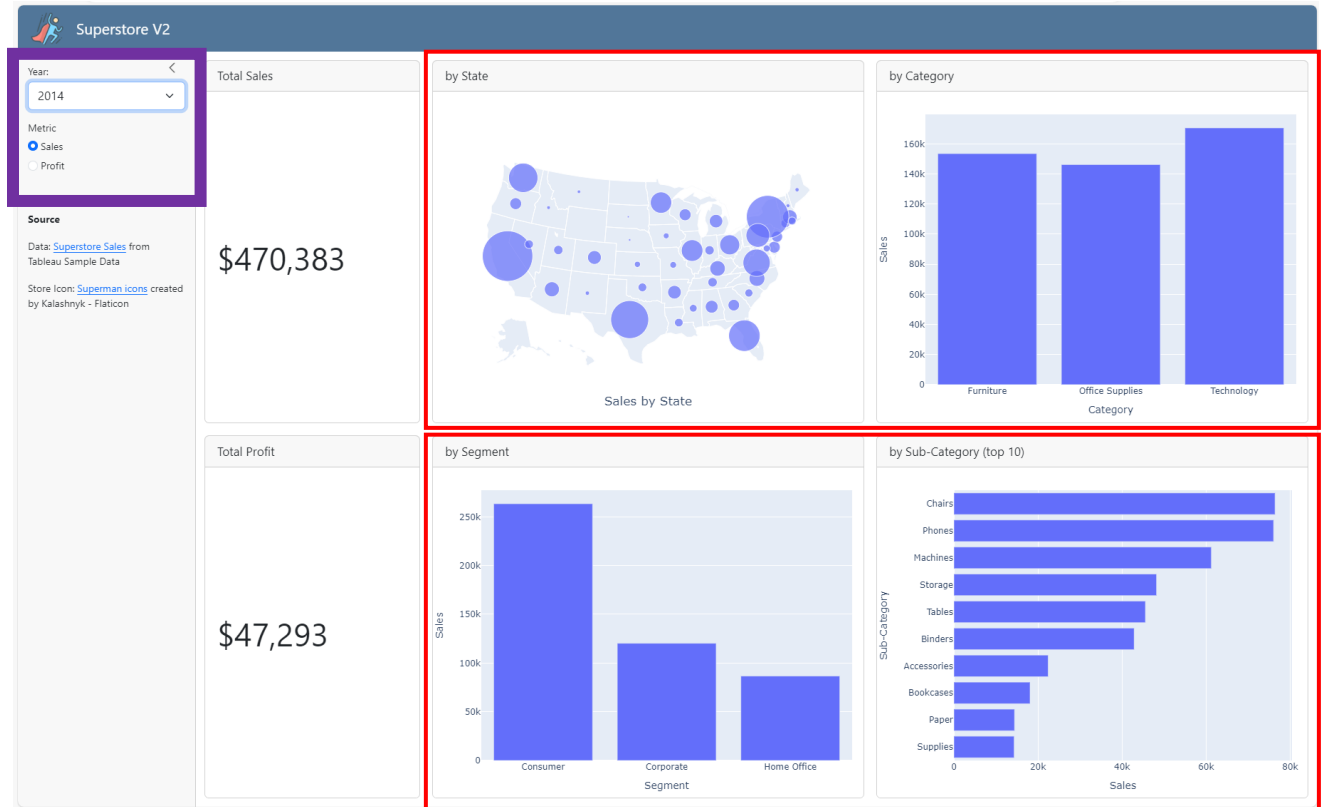
```
```{python}
#| title: by Segment
#| padding: 0
```

```
→@render_widget
def plot_by_segment()
```
```

```
## Column {width=40%}
```

```
```{python}
#| title: by Category
#| padding: 0
```

```
```{python}
#| title: by Sub-Category (top 10)
#| padding: 0
```



- > quarto render superstore-v2.qmd
- > shiny run app.py

Deploy Your Dashboard

- Simple (static) dashboard
 - Any cloud services that can host websites
 - e.g., [Github](#), [Posit Connect Cloud](#), etc.
 - Document: Deploy to [Github](#), [Posit Connect Cloud](#) (and [more](#))
 - Of course you can setup your own web server too
 - You can use `quarto publish` command to make the deployment easy
- Enhanced (shiny) dashboard
 - Publishing services that supports shiny app
 - e.g., [Posit Connect Cloud](#) and [Shiny on Space](#) from [Hugging Face](#)
 - Document: Deploy to Posit Connect Cloud (as of Aug 22, 2025, need to [deploy as a shiny for Python app](#); Deploying as a Quarto dynamic dashboard [added in Dec 2025](#);), [Shiny on Space](#)
 - You can also setup your own [shiny server](#) to host your dashboard app

Posit Connect Cloud

- Posit's next-generation online publishing platform
 - Support Quarto document, Shiny for Python app, Streamlit app, etc.
- Easy to use
 - Deploy a document or an app by pointing to its Github repo
 - Alternatively, deploy via Quarto CLI (command line) or an IDE such as VS Code, Positron, or RStudio